

GradPromptOpt: An enhanced prompt optimization method to improve performance of LLMs

Jianlin Zhong

Yzw.AI.Center

Yzw.cn

Chengdu, CHN

Jalen.Zhong@outlook.com

Manchang Gu

Yzw.AI.Center

Yzw.cn

Chengdu, CHN

manchanggu@gmail.com

Zhaowen Tao

Yzw.AI.Center

Yzw.cn

Chengdu, CHN

taozhaowen@gmail.com

Dong Tang

Yzw.AI.Center

Yzw.cn

Chengdu, CHN

dongai01@163.com

Mingjiang Xie

Yzw.AI.Center

Yzw.cn

Chengdu, CHN

Xiemingjiang1997@gmail.com

Zhengyu Zhang*

Yzw.AI.Center

Yzw.cn

Chengdu, CHN

* Corresponding author: Zhangzhenyu13@outlook.com

Abstract—In the application of Large Language Models (LLMs), the Prompt plays a crucial role as a key component influencing model performance. A well-crafted Prompt can significantly enhance an LLM's ability to perform specific tasks effectively. LLMs process Prompts by converting them into preceding tokens, which are then used to predict subsequent tokens. This process implicitly and uniquely translates human language into the model's semantic space—where even semantically similar Prompts can lead to vastly different outputs due to subtle variations in phrasing or structure. To mitigate the uncertainty introduced by diverse human language expressions and improve the adaptability of LLMs to various Prompts, we propose a novel method called GradPromptOpt (Gradient-Driven Prompt Optimization). This approach leverages the gradient-based optimization mechanisms inherent in LLMs and employs a carefully fine-tuned Meta Prompt to automate and refine the Prompt optimization process. GradPromptOpt aims to surpass the effectiveness of traditional, manually designed Prompts while reducing the reliance on subjective human input. Experimental results show that GradPromptOpt achieves a 31.75% performance improvement over high-quality, human-engineered Prompts, demonstrating its potential to enhance the efficiency and adaptability of LLMs in practical applications.

Keywords—LLM, Prompt Optimization, Gradient Mechanism, Artificial Intelligence

I. INTRODUCTION

Large language models (LLMs) trained on vast amounts of web data have demonstrated breakthrough capabilities in natural language processing (NLP) [1]. Through instruction-based prompting mechanisms, these models achieve versatile human-computer interaction, enabling tasks such as question answering, summarization, code generation, and complex reasoning [2]. Despite their impressive performance, existing prompt

engineering methods remain heavily reliant on manual tuning processes, which demand significant human expertise and labor costs. This dependency not only limits scalability but also hinders adaptability across diverse applications. Consequently, developing intelligent prompt generation systems has emerged as a critical research direction. Such systems promise to reduce manual intervention, enhance task-specific prompt quality, and provide interpretable analytical pathways for model behavior refinement [3][4].

To address these challenges, recent studies have explored automated prompt optimization frameworks. Approaches like APE [5] and APPO [6] employ black-box optimization and reinforcement learning, respectively, to reduce manual effort. However, these methods often face limitations in precision and generalizability. For instance, gradient-free optimization struggles to identify fine-grained prompt weaknesses, while reinforcement learning-based methods require complex reward engineering. Against this backdrop, we propose GradPromptOpt, a novel gradient-informed automatic prompt optimization framework. By integrating gradient signals derived from both positive and negative samples, our method enables precise identification of prompt weaknesses and systematic refinement through iterative editing. This two-stage process—evaluation and editing—generates "natural language gradients" that guide LLMs toward task-aligned prompt improvements, significantly enhancing optimization efficiency and interpretability.

II. RELATED WORKS

Recent advancements in prompt optimization focus on reducing reliance on manual engineering. APE [5] redefines prompt generation as a black-box optimization problem, using LLMs to iteratively propose and select high-performing prompts. While effective for specific tasks, its search process lacks

gradient-informed guidance, limiting efficiency in complex scenarios. APPO [6] introduces reinforcement learning with adaptive Proximal Policy Optimization (PPO) to stabilize training and expand the search space. Though robust, its dependency on reward modeling complicates deployment. BPO [8] offers a lightweight alternative by optimizing user inputs instead of model parameters, but its input-centric approach may overlook deeper prompt-task misalignments.

A promising direction involves leveraging LLMs as gradient estimators. ORPO [7] pioneers this concept by constructing meta-prompts from solution-score pairs to guide optimization, bypassing explicit gradient computation. However, its reliance on heuristic scoring limits adaptability. In contrast, ProTeGi [9] explicitly frames LLMs as gradient estimators but restricts analysis to negative samples, which our experiments reveal as suboptimal. Negative samples alone fail to provide directional guidance for improvement, whereas incorporating positive-negative sample pairs enables clearer identification of optimization trajectories.

These insights motivate our GradPromptOpt framework, which advances gradient-based prompt optimization in two ways: (1) By utilizing LLMs to estimate gradients from both positive and negative examples, it captures bidirectional improvement signals; (2) By translating gradients into natural language feedback, it enables interpretable, stepwise prompt refinement. This approach addresses the limitations of prior methods while maintaining compatibility with diverse LLMs and tasks.

III. PROPOSED METHOD

A. Framework Architecture

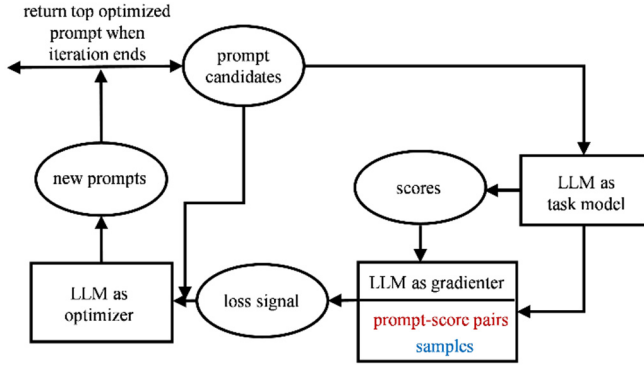


Figure 1. Framework architecture of GradPromptOpt

The Gradient-Driven Prompt Optimization (GradPromptOpt) framework, as depicted in Fig. 1, innovatively employs Large Language Models (LLMs) to serve dual roles as both optimizers and gradient estimators. This approach enables the iterative refinement of prompts tailored for specific downstream tasks. By establishing a closed-loop system, GradPromptOpt can continuously adjust and improve these prompts based on feedback loops, leading to enhanced task performance. Essentially, this framework harnesses the power of LLMs to streamline and refine the prompt optimization process, ensuring

more effective and efficient outcomes for various applications. The optimization trajectory follows:

$$P^{(t+1)} = \Gamma(P^{(t)} | \Theta_{LLM}, F_{task}) \quad (1)$$

where $P^{(t)}$ denotes the prompt population at iteration t , Θ_{LLM} represents LLM parameters, and F_{task} symbolizes task-specific feedback.

B. Initialization of Prompt Candidates Framework Architecture

The process begins with a diverse set of initial prompt candidates, which can be manually designed, generated by applying random perturbations to base prompts, or produced by an LLM based on task descriptions. This diversity ensures broad exploration of the prompt space. Mathematically, the step of initialization prompt p can be represented as:

$$p = \{p_i | p_i \in P, i \in [1, n]\} \quad (2)$$

where P denotes the universal set of all possible prompts, and n represents the number of initial candidates. This ensures that the optimization process starts with a wide range of possibilities, enabling effective exploration of the solution space.

C. Score Evaluation

Each prompt candidate is evaluated on the target task to compute a score within the range of 0-1. The resulting prompt-score pairs form a dataset O , which serves as the basis for gradient estimation. The scoring function $S(P)$ is defined as a binary indicator function those outputs 1 if the prompt's output matches the desired answer, and 0 otherwise:

$$S(P) = \frac{\sum_{i=0}^n \mathbb{I}(O(p_i) = a_i)}{n} \quad (3)$$

$$\mathbb{I}(x) = \begin{cases} 1, & \text{if } x \text{ is true} \\ 0, & \text{otherwise} \end{cases} \quad (4)$$

where $O(p_i)$ represents the output generated by the model using the prompt p_i , a_i is the desired answer. This evaluation step ensures that each prompt is quantitatively assessed, providing a foundation for subsequent optimization.

D. Gradient Estimation Using LLMs

In this stage, the LLM functions as a gradient estimator, inferring direction for prompt updates by analyzing differences between high-scoring and low-scoring prompts. The model identifies patterns in high-scoring prompts (such as effective keywords or structures), compares them with lower-scoring counterparts, and generates gradient-like rules (e.g., "replace vague verbs with specific actions"). This step mimics gradient calculation in continuous optimization, where the LLM maps score differences to semantic edits through text reasoning.

$$\Delta p \approx \nabla_s \mathcal{L}(p) \approx \operatorname{argmax}_p \langle LLM(D), (P, \Delta S, E) \rangle \quad (5)$$

$$E = \{O(p), a\} \quad (6)$$

Where $\mathcal{L}(p)$ represents the loss function associated with the prompt p , and $\langle \cdot, \cdot \rangle$ denotes the inner product capturing the alignment between the LLM's inferred rules and the Optimization-targeted prompt, the score gradient ΔS , and the example pairs. This mechanism enables the framework to effectively leverage the LLM's understanding of language semantics, producing impactful suggestions to guide the optimization.

E. Prompt Optimization Using LLMs

Utilizing the inferred gradient rules, the LLM acts as an optimizer to refine the prompts. It takes the highest-scoring candidate and its corresponding gradient rules as input to generate new prompts aligned with the desired update direction. The optimization process is not merely a direct substitution but involves a sophisticated balance between adhering to the inferred gradient direction and maintaining linguistic coherence. This ensures that the refined prompts remain semantically meaningful while progressively improving their performance on the target task. The optimization step can be expressed as:

$$\hat{p} = \operatorname{argmin}_p \| p - (p^* + \eta \Delta p) \|^2 \quad (7)$$

where $\hat{p}$ is the optimized prompt, p^* is the highest-scoring prompt of last iteration, η is the learning rate. Furthermore, the iterative nature of this refinement process allows for continuous improvement. The process runs for n iterations, at each iteration, the LLM generates a new prompt based on the updated gradient rules, which are then re-evaluated to compute their scores. This feedback loop ensures that the optimization process converges toward prompts that not only achieve high scores on the target task but also maintain clarity and coherence.

The above steps are iterated until convergence or a predefined iteration limit is reached. During each iteration, new prompts are added to the candidate pool, low-scoring candidates are discarded to maintain computational efficiency, and the top- k prompts are retained for the next cycle. The final output is the highest-scoring prompt from the last iteration, achieving optimal task performance.

By framing prompt optimization as a gradient-driven process, GradPromptOpt bridges discrete text generation with continuous optimization paradigms. The dual role of LLMs as gradient estimators and optimizers enables systematic exploration of the prompt space under task-specific feedback guidance, without requiring differentiable tokenization tools or embeddings. This approach is model-agnostic and applicable to a wide range of natural language processing tasks.

IV. EXPERIMENTAL RESULTS & ANALYSIS

Our objective is to validate the performance of the GradPromptOpt framework under various conditions through a series of experiments. These include assessing the importance of

gradient estimation, comparing the effectiveness of different types of prompts, analyzing the impact of varying LLM scales, and exploring how the number of positive and negative samples affects optimization outcomes. These experiments will help us gain a comprehensive understanding of the advantages and applicable scenarios of the GradPromptOpt framework, providing valuable insights for future research.

In this study, we employ a custom dataset—Material Information Keyword Extraction (MIKE)—to evaluate the efficacy of the GradPromptOpt framework. The MIKE dataset involves accurately extracting keywords related to materials from a series of text descriptions. For instance, given the input "50mm acoustic testing pipe procurement and installation," the expected output would be "acoustic testing pipe." This type of task is highly sensitive to prompts, often requiring meticulous fine-tuning by human experts to achieve satisfactory results. Given this characteristic, we believe the MIKE dataset is particularly well-suited for demonstrating the advantages of the GradPromptOpt framework.

A. Ablation Compare

To evaluate the importance of gradient estimation within the GradPromptOpt framework, we conducted a series of ablation experiments. These experiments were designed to rigorously assess the impact of gradient estimation on model performance across a diverse range of NLP tasks. We selected four distinct datasets—LIAR, Sarcasm, Ethos, and MIKE—each representing unique task scenarios, including lie detection, sarcasm recognition, ethical judgment, and keyword extraction. This selection was motivated by the desire to validate both the effectiveness and generalizability of gradient estimation in optimizing prompts for a wide variety of linguistic and reasoning challenges.

The experimental setup involved comparing two configurations: one where gradient estimation was utilized (denoted as "w gradient") and another where it was omitted (denoted as "w/o gradient"). By systematically removing gradient estimation, we aimed to isolate its contribution to the overall performance of the GradPromptOpt framework. The results, presented in the table below, provide compelling evidence of the critical role played by gradient estimation:

Table 1 Ablation experiment

	LIAR	sarcasm	ethos	Mike
w/o gradient	0.71	0.87	0.95	0.80
w gradient	0.74	0.92	0.96	0.83

As can be seen from TABLE 1 above, the GradPromptOpt method with gradient estimation achieved significant performance improvements over the method without gradient estimation across all tested datasets. This indicates that gradient estimation plays a crucial role in prompt optimization.

B. Prompt Compare

We selected the "MIKE" dataset for a case study, comparing the original human-crafted prompts, the results of optimized human prompts, and the target prompts directly generated by GradPromptOpt. This comparison allows us to explore

GradPromptOpt's capabilities in both refining existing prompts and generating new prompts from scratch. Such a setup helps us understand whether GradPromptOpt can further enhance model performance based on existing prompts or if it requires high-quality initial prompts to be effective. The specific results are as follows:

Table 2 Experiment of prompt compare

	Mike
Human Prompt	0.63
GradPromptOpt with human prompt	0.83
GradPromptOpt with destination	0.64

As shown in TABLE 2 the most striking result is that optimizing human-designed prompts using GradPromptOpt leads to a significant improvement in model performance, raising the score from 0.63 to 0.83. This demonstrates the tool's ability to refine and enhance existing prompts, leveraging its optimization algorithms to identify subtle improvements that humans may overlook. It suggests that even well-crafted human prompts can benefit substantially from automated refinement. When GradPromptOpt generates target prompts entirely on its own (without relying on human input), the resulting performance score is 0.64, which is marginally higher than the baseline human-crafted prompts (0.63). While this indicates that GradPromptOpt has some capability to generate functional prompts independently, the results suggest that direct generation may not yet match the quality achieved through iterative optimization of pre-existing prompts. The results imply that GradPromptOpt performs best when provided with high-quality initial prompts. In cases where no such foundation exists, the system still produces usable outputs but falls short of the exceptional gains seen with refined prompts. This highlights the importance of combining human expertise with algorithmic optimization for optimal results.

C. LLM Compare

We employed Qwen2.5 series large language models (LLMs) of varying scales—7 billion, 14 billion, and 72 billion parameters [10]—to rigorously test their efficacy as both optimizers and gradient estimators. Our hypothesis was grounded in the theoretical advantage that larger models possess enhanced capabilities for text understanding and generation due to their increased complexity and capacity to capture finer nuances within the data. By assessing models across this spectrum, our methodology aimed not only at identifying the most effective model configuration but also at meticulously observing how performance trends evolve with an increase in model scale. The results are presented in the table below:

Table 3 Experiment of LLM scale compare

LIAR		Task Model		
		Qwen2.5-7b	Qwen2.5-14b	Qwen2.5-72b
Optimizer Model + Gradient Model	Qwen2.5-7b	0.7188	0.7344	0.7031
	Qwen2.5-14b	0.6979	0.7083	0.7262
	Qwen2.5-72b	0.7292	0.7344	0.7422

The experiments TABLE 3 demonstrate that increasing the scale of LLMs generally leads to improved performance, with Qwen2.5-72B emerging as the most effective model across all configurations. While smaller models can still contribute meaningfully, especially in hybrid setups, the superior capabilities of larger models make them ideal candidates for both optimizer and gradient roles. These findings underscore the importance of leveraging large-scale models in prompt engineering and highlight the potential for further advancements in LLM research through targeted optimizations and efficient scaling strategies.

D. Parameter Compare

Finally, we investigated the impact of the number of positive and negative samples on the performance of GradPromptOpt for the MIKE task. The quantity and quality of training data typically have a direct influence on the model's learning ability and generalization performance. By systematically varying the number of positive and negative examples and recording their effects on model scores, we can better understand how to balance the ratio of positive to negative samples for optimal prompt optimization. The specific results are as follows:

Table 4 Experiment of example parameter compare

Gradient + MIKE		Positive Example		
		0	5	10
Nagetive Example	0	-	0.65	0.80
	5	0.55	0.66	0.81
	10	0.53	0.68	0.83

The experiment TABLE 4 revealed a trend of improved model performance as the number of positive and negative samples increased. Notably, the model achieved its highest score when both positive and negative samples were at their maximum (10 each). This indicates that an adequate number of positive and negative samples can significantly enhance model performance and underscores the importance of data quantity in model optimization.

V. CONCLUSION

The rapid advancements in large language models (LLMs) have revolutionized natural language processing, enabling remarkable capabilities in human-computer interaction across diverse tasks. However, the reliance on manual prompt engineering has posed significant challenges in terms of scalability, adaptability, and cost-efficiency. To address these limitations, research efforts have increasingly focused on developing intelligent and automated prompt optimization systems. Approaches such as APE, APPO, ORPO, and BPO have demonstrated promising results by leveraging techniques like black-box optimization, reinforcement learning, and input refinement to reduce human intervention and enhance prompt quality. Building on these foundations, GradPromptOpt introduces a novel gradient-informed method that refines prompts with greater precision and scalability. By integrating "natural language gradients" to guide iterative improvements, GradPromptOpt not only aligns prompts more effectively with target tasks but also streamlines the optimization process. This advancement represents a significant step toward more efficient,

interpretable, and scalable prompt engineering, paving the way for broader adoption of LLMs across various applications while reducing dependency on manual tuning and domain expertise.

Despite the advancements presented, our work is not without its limitations, and several promising avenues for future research could further enhance GradPromptOpt. One potential direction involves integrating multi-modal data, such as images or audio, into the optimization process. This expansion would empower the system to address tasks that extend beyond conventional text-based NLP, opening new horizons for application. It is noteworthy, however, that our current exploration has not ventured into experiments with multi-modal large language models (LLMs) for automatic prompt optimization. This oversight represents a significant gap in our study, especially considering the growing prevalence of multi-modal LLMs in real-world applications. Optimizing prompts for these advanced models holds the promise of unlocking enhanced capabilities and elevating performance across a diverse array of tasks. Future investigations should therefore focus on extending GradPromptOpt to accommodate multi-modal inputs, thereby broadening its applicability and ensuring its relevance to an ever-evolving landscape of technological possibilities. By doing so, we can continue to push the boundaries of what is achievable with intelligent prompt generation systems.

While our experiments have demonstrated the effectiveness of GradPromptOpt within the Qwen2.5 series of large models, this focus may introduce certain limitations. Specifically, testing the method exclusively on one family of models might not fully capture its generalizability or robustness when applied to other architectures or model types. For instance, different LLMs may exhibit varying sensitivities to prompt structures or optimization strategies, which could influence the efficacy of GradPromptOpt. Therefore, future research should aim to validate the approach across a wider variety of model families and sizes, including both open-source and proprietary models, to ensure their broad applicability and reliability.

Additionally, efforts to integrate explainability features into GradPromptOpt could provide users with deeper insights into the optimization process, fostering trust and transparency in AI-driven decision-making. Such enhancements would not only make the tool more interpretable but also enable researchers and practitioners to better understand why specific prompts perform well or poorly. By addressing these areas—extending GradPromptOpt to multi-modal settings, validating its performance across diverse model architectures, and enhancing its explainability—we can pave the way for a more versatile, reliable, and user-friendly prompt optimization framework.

ACKNOWLEDGMENT

The authors would like to express their sincere gratitude for the computational resources and data support generously provided by YZW Company. Their contribution has been crucial to the completion of this work. Additionally, we would like to thank our team members for their tireless efforts in conducting surveys and providing experimental assistance. This research would not have been possible without their dedication and hard work.

REFERENCES

- [1] J. Achiam, S. Adler, S. Agarwal, et al., "GPT-4 technical report," arXiv preprint arXiv:2303.08774, 2023.
- [2] D. Guo, D. Yang, H. Zhang, et al., "DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning," arXiv preprint arXiv:2501.12948, 2025.
- [3] J. D. Zamfirescu-Pereira, R. Y. Wong, B. Hartmann, et al., "Why Johnny can't prompt: How non-AI experts try (and fail) to design LLM prompts," in Proc. 2023 CHI Conf. Human Factors in Computing Systems, 2023, pp. 1-21.
- [4] L. Reynolds and K. McDonell, "Prompt programming for large language models: Beyond the few-shot paradigm," in Ext. Abstracts of the 2021 CHI Conf. Human Factors in Computing Systems, 2021, pp. 1-7.
- [5] Y. Zhou, A. I. Muresanu, Z. Han, et al., "Large language models are human-level prompt engineers," arXiv preprint arXiv:2211.01910, 2022.
- [6] M. R. Kwon, G. Kim, J. Kim, et al., "StablePrompt: Automatic Prompt Tuning using Reinforcement Learning for Large Language Models," arXiv preprint arXiv:2410.07652, 2024, in press.
- [7] Y. Cheng, X. Wang, Y. Lu, H. Liu, Q. V. Le, D. Zhou, and X. Chen, "Large language models as optimizers," in Proc. International Conference on Learning Representations (ICLR), 2024.
- [8] J. Cheng, X. Liu, K. Zheng, et al., "Black-box prompt optimization: Aligning large language models without model training," in Proc. Annual Meeting of the Association for Computational Linguistics (ACL), 2024.
- [9] R. Pryzant, D. Iter, J. Li, et al., "Automatic prompt optimization with 'gradient descent' and beam search," in Proc. Conference on Empirical Methods in Natural Language Processing (EMNLP), 2023.
- [10] A. Yang, B. Yang, B. Zhang, et al., "Qwen2.5 technical report," arXiv preprint arXiv:2412.15115, 2024.